

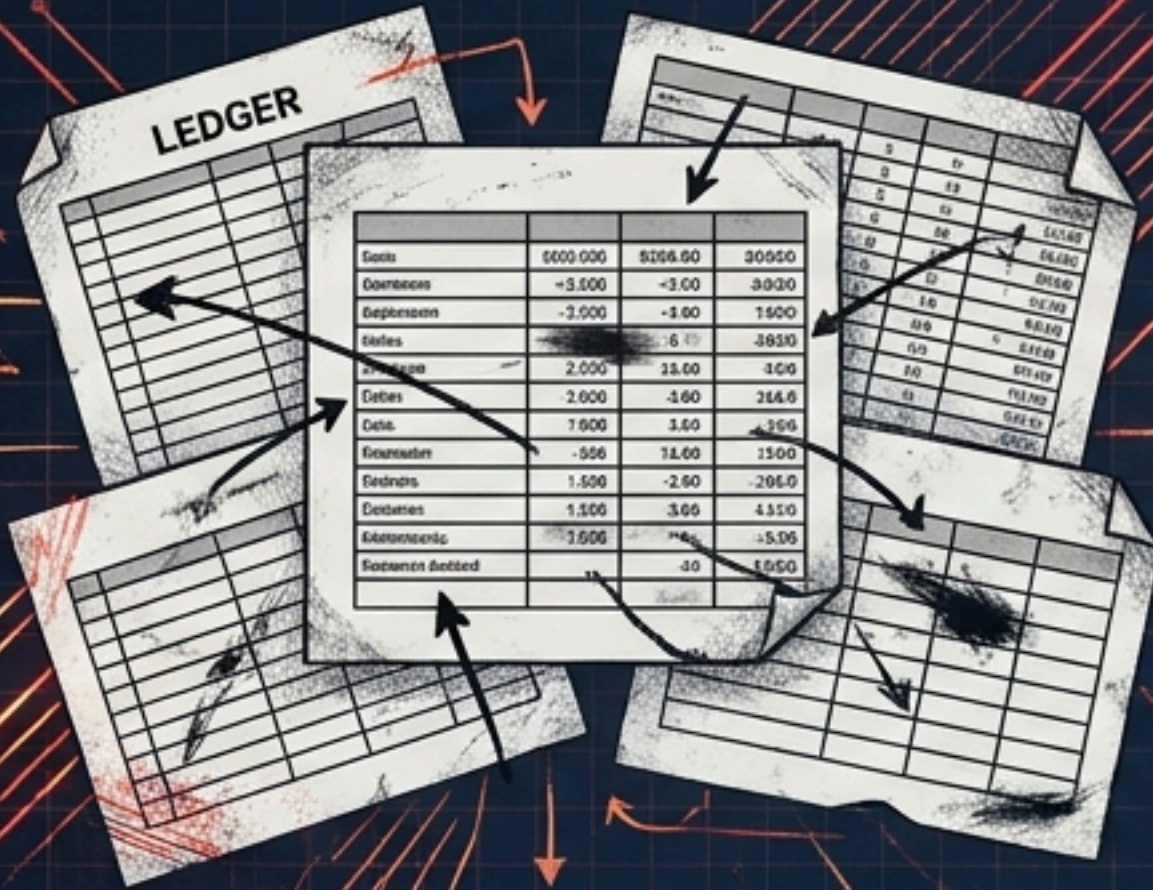
PERSONAL FINANCE TRACKER

Academic Project Defense

Project Scope: Case 3
Course: **Introduction** to Programming 2 (Python)

Yussupov Nurlan
Asylzhan Amangeldi
Bakdaulet Begaliev
Dias Sabit

The Limitations of Manual Ledgers



Assets	\$600,000	\$126.60	30050
Overhead	-5,500	-3.00	3000
Depreciation	-3,000	-1.80	1500
Debits		-6.80	3630
Overhead	2,000	12.00	300
Debits	2,000	-1.80	264.6
Cash	1,000	3.60	356
Revenue	-556	12.00	1500
Debits	1,500	-2.80	206.0
Debits	1,500	3.60	4350
Revenue	1,500		-5.06
Revenue		30	1050

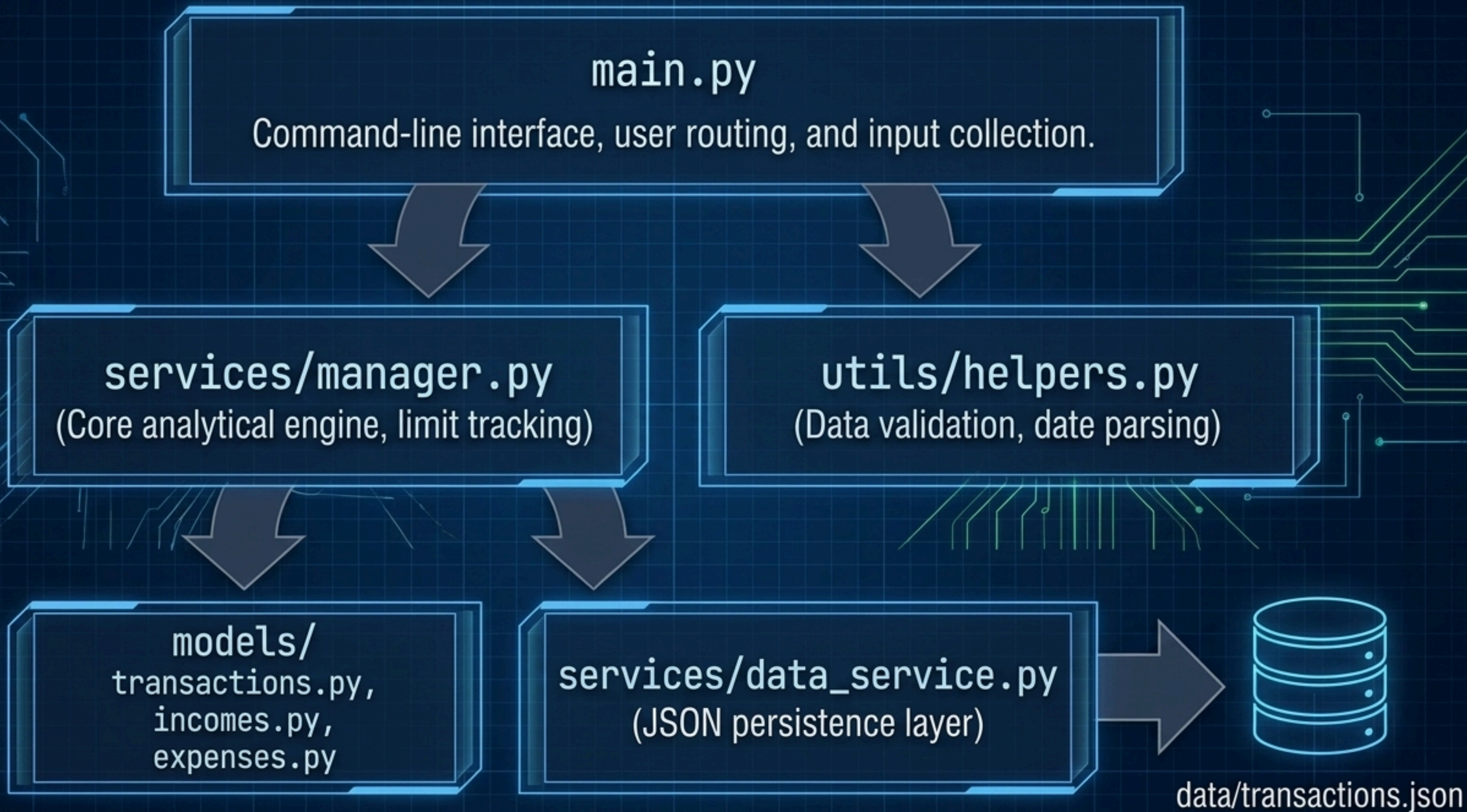
- Uncalculated totals lead to missed overspending.
- Scattered data prevents accurate category breakdowns.
- Time-consuming monthly roll-ups.

Automated Tracking System

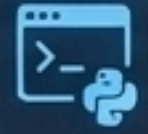


- Instant, algorithmic monthly summaries (YYYY-MM).
- Automated expense category grouping.
- Configurable, real-time overspending detection limits.

Moving from static records to dynamic financial analysis



Core Data Models & Polymorphism



Protected attributes (`_amount`, `_date`) strictly controlled via `@property` decorators to prevent invalid state manipulation.

Transaction

Income

Expense

Polymorphism

Enables single-pass balance calculation:
`sum(t.sign_amount() for t in self.transactions)`

```
1 # Income Class
2 def sign_amount(self):
3     return self.amount
4
5 # Expense Class
6 def sign_amount(self):
7     return -self.amount
```


The Interface

1. View Current Balance
2. Add Income
3. Add Expense
4. View Category Breakdown & Stats
5. View Monthly Summary
6. Check for Overspending
7. Exit

Select an option (1-7): |

The Persistence Layer

```
[  
  {  
    "amount": 250000.0,  
    "date": "2026-05-01",  
    "type": "income",  
    "source": "salary"  
  }  
]
```


Statistical Grouping

```
Unique categories: 3
Categories: food, rent, transport
- Food: 23500 tenge
- Rent: 80000 tenge
- Transport: 6500 tenge
```

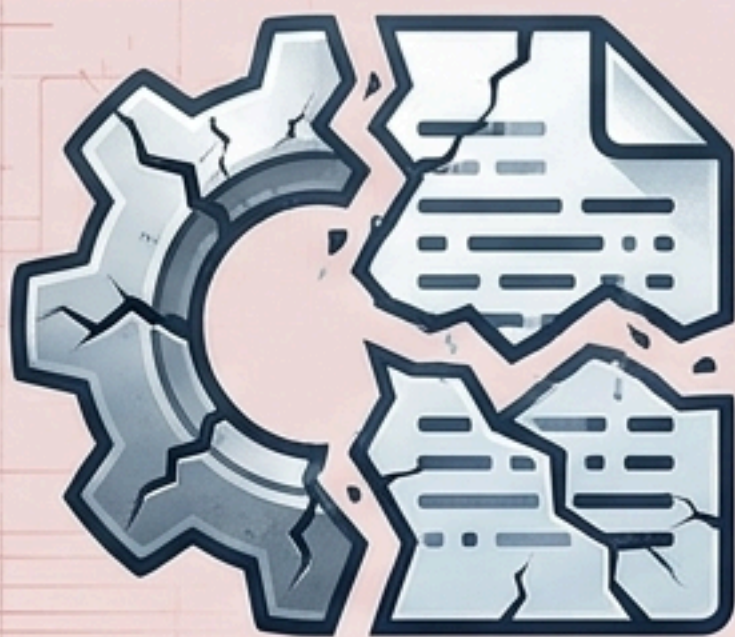
Utilizes Python dictionaries for $O(1)$ category lookups and updates:
`result[category] += amount`

Temporal Filtering

```
--- Monthly Summary for 2026-05 ---
Total Income: 265000 tenge
Total Expenses: 136500 tenge
Month Balance: 128500 tenge
```

Aggregates temporal data using high-order `filter()` and `map()` functions.

Standard Library Constraint



Python's native json module cannot serialize custom OOP objects (Income and Expense classes) out of the box, leading to serialization crashes on exit.

Dynamic Serialization Loop (Dias's Architecture)



```
1 for t in transactions_list:
2     if hasattr(t, "get_details"):
3         prepared_list.append(t.get_details())
4     elif isinstance(t, dict):
5         prepared_list.append(t)
6     else:
7         prepared_list.append(t.__dict__)
```



Implemented
`os.makedirs(os.path.dirname(DATA_FILE),
exist_ok=True)` to dynamically generate
database directories on first launch,
preventing `FileNotFoundError`.



Code Evidence (Focus on Date Validation)

```
try:  
    datetime.strptime(date_txt, "%Y-%m-%d")  
    return date_txt  
except ValueError:  
    raise ValueError("Дата должна быть YYYY-MM-DD")
```

Strict type enforcement ensures the analytical engine never receives malformed data.

Defense Roster

Team Member	Core Git Contributions	Defense Domain
 Nurlan Yussupov	Core Manager logic, financial statistics, overspending limit tracking, and unittest development.	Analytical calculations, limits tracking, algorithmic logic.
 Asylzhan Amangeldi & Bakdaulet Begaliev	Core transaction model development, calculation integration, and data validation support.	Transaction model hierarchies, object structures, class properties.
 Dias Sabit	Data Layer (data_service.py), custom JSON serialization, Git conflict resolution, input validation integration.	Persistence layer, object serialization, system error handling workflows.

Academic Evaluation Scorecard



Strict OOP Paradigm

4 distinct classes utilizing proper Encapsulation, Inheritance, and Polymorphism.



Optimized Python Collections

Dictionaries: Used for $O(1)$ statistical updates. Sets: Enforce unique category lists.
Tuples: Lock immutable report rows.



Robust Reliability

Zero-crash terminal experience driven by comprehensive try-except data validation.



Verified Code Quality

Fully integrated unittest suite (10 passing tests) ensuring calculation integrity.

A modular, scalable, and resilient approach to automated financial tracking.